

MINISTERE DE L'ENSEIGNEMENT
SUPÉRIEUR, DE LA RECHERCHE
ET DE L'INNOVATION

INSTITUT BURKINABE DES ARTS
ET METIERS

BURKINA FASO

La patrie où la mort nous vaincrons.



**Conception et mise en œuvre d'une plateforme web de
gestion des mémoires avec moteur intégré de détection
de plagiat**

Par : DICKO Alou

Enseignant : Dr Yacouba Ouattara

TABLES DES MATIERES:

INTRODUCTION.....	3
CHAPITRE I : ANALYSE ET CONCEPTION.....	4
I. Étude préalable et Besoins.....	4
1. Analyse de l'existant et critique.....	4
2. Spécification des besoins fonctionnels.....	5
3. Spécification des besoins non-fonctionnels.....	6
II. Modélisation UML (Approche 2TUP).....	6
1. Capture des besoins fonctionnels (Cas d'utilisation).....	6
Figure 1: Diagramme de cas d'utilisation.....	7
2. Analyse dynamique (Diagrammes de séquence).....	8
Figure 2: Diagramme de Séquence : Processus de soumission et validation de thème.....	10
Figure 3: Diagramme de Séquence : Processus de dépôt et d'analyse automatique de mémoire.....	10
3. Analyse statique (Diagramme de classes).....	10
Figure 4: Diagramme de classe.....	12
CHAPITRE II : RÉALISATION ET MISE EN ŒUVRE.....	13
I. Environnement de développement.....	13
1. Choix technologiques.....	13
2. Environnement matériel et logiciel.....	13
II. Implémentation des fonctionnalités.....	14
1. Développement du moteur de détection (Plagiat & IA).....	14
2. Gestion du workflow de dépôt et validation.....	15
III. Tests et Sécurité.....	15
1. Plan de tests (Unitaires et Intégration).....	15
2. Mesures de sécurité et protection des données.....	16
CONCLUSION.....	17

INTRODUCTION

Le projet Handal s'inscrit dans un contexte académique où l'intégrité intellectuelle représente un enjeu fondamental pour la crédibilité des institutions d'enseignement supérieur. Avec l'évolution constante des outils numériques et la diversification des méthodes de génération de contenu, la protection de l'originalité des travaux de recherche est devenue une priorité. Cette plateforme, conçue avec la technologie Next.js, propose une solution intégrée pour la gestion des mémoires associée à un système de détection de similitudes. L'objectif est de sécuriser le patrimoine scientifique de l'établissement tout en modernisant les interactions entre les différents acteurs de la chaîne académique.

La problématique centrale réside dans l'obsolescence des méthodes de gestion traditionnelles face aux nouveaux défis du numérique. Les processus actuels, souvent manuels, peinent à garantir une vérification systématique et fiable de l'authenticité des documents soumis. La complexité des formes de fraude modernes, comme la reformulation ou la traduction de sources étrangères, rend les contrôles humains insuffisants. Par ailleurs, l'absence de centralisation des flux de travail crée des lenteurs administratives et un manque de visibilité sur l'avancement des dossiers, compromettant ainsi l'efficacité globale du suivi pédagogique.

Pour répondre à ces enjeux, le projet Handal se fixe des objectifs de transformation profonde. Il s'agit avant tout de digitaliser l'intégralité du parcours du mémoire, depuis la validation du sujet jusqu'à la phase finale d'archivage. Le système vise à instaurer une détection intelligente capable d'identifier les similitudes textuelles, garantissant ainsi une plus grande équité entre les étudiants. En plus de renforcer l'intégrité académique, le projet ambitionne d'optimiser la collaboration entre les étudiants, les encadrants et l'administration grâce à une interface unifiée et des notifications automatiques.

Afin de détailler la conception et la réalisation de ce projet, ce rapport est organisé en quatre axes principaux. Nous commencerons par l'analyse et la conception du système, incluant l'étude des besoins et la modélisation des processus. Ensuite, nous présenterons l'architecture technique et les choix technologiques qui soutiennent la plateforme. Le troisième axe sera consacré au développement du moteur de détection et aux fonctionnalités clés de l'application. Enfin, nous aborderons les phases de test, de déploiement et les perspectives d'évolution pour assurer la pérennité de la solution.

CHAPITRE I : ANALYSE ET CONCEPTION

I. Étude préalable et Besoins

1. Analyse de l'existant et critique

L'organisation actuelle de la gestion des mémoires au sein de l'institution repose sur des processus fragmentés et essentiellement manuels. Le circuit de validation des documents dépend d'échanges physiques ou de communications par courriels qui ne sont pas centralisés dans un système d'information dédié. Cette situation oblige les étudiants et les enseignants à multiplier les démarches administratives pour assurer le suivi de chaque étape du travail de recherche. En l'absence d'un point d'entrée unique pour le dépôt et la correction des fichiers, l'administration éprouve des difficultés à maintenir une vision claire et en temps réel de l'état d'avancement des promotions.

La critique de ce mode de fonctionnement met en lumière des vulnérabilités importantes en matière de gestion documentaire. Le manque de coordination numérique entre les intervenants génère des délais de traitement importants et augmente le risque de perte d'informations critiques. De plus, la sécurité des données n'est pas pleinement assurée, car les documents circulent sur des supports variés sans contrôle d'accès rigoureux ni historique des modifications. Cette opacité administrative rend difficile l'auditabilité des processus, empêchant ainsi de retracer avec précision les validations successives qui jalonnent le parcours académique de l'étudiant.

Sur le plan pédagogique, le système actuel ne dispose pas de mécanismes automatisés pour vérifier l'intégrité des contenus. La détection des cas de plagiat repose uniquement sur l'expertise et la vigilance des encadrants, ce qui introduit une part de subjectivité et laisse passer des fraudes de plus en plus sophistiquées. Cette lacune technique affaiblit la capacité de l'institution à garantir l'originalité des travaux produits sous son égide. L'absence d'un outil de comparaison systématique avec des bases de données de référence crée un déséquilibre dans l'évaluation de la qualité scientifique, rendant indispensable la mise en place d'une solution capable d'automatiser ces contrôles de manière objective et transparente.

2. Spécification des besoins fonctionnels

La plateforme Handal doit avant tout assurer une gestion rigoureuse des accès à travers un système d'authentification sécurisé et une gestion des rôles adaptée aux différents profils d'utilisateurs. Les administrateurs, les enseignants et les étudiants doivent disposer d'interfaces spécifiques leur permettant d'interagir avec le système selon leurs prérogatives respectives. Cette centralisation des comptes garantit que chaque acteur n'accède qu'aux informations et aux fonctionnalités qui lui incombent, assurant ainsi la confidentialité des recherches et la sécurité des processus de validation dès l'entrée sur la plateforme.

Au cœur des besoins fonctionnels se trouve l'automatisation complète du cycle de vie du mémoire, commençant par la soumission en ligne des thèmes de recherche. Le système doit permettre aux étudiants de déposer leurs propositions, lesquelles sont ensuite soumises à un circuit de validation électronique par les instances académiques. Tout au long de la phase de rédaction, la plateforme doit offrir des outils de suivi permettant de consigner les versions successives du document et de faciliter les échanges entre l'étudiant et son encadrant. Ce workflow structuré remplace les communications informelles par un journal de bord numérique, offrant une visibilité en temps réel sur l'état d'avancement de chaque projet de recherche.

L'un des piliers majeurs de la solution réside dans sa capacité de détection intelligente des similitudes. Le système doit être capable d'analyser automatiquement chaque document déposé en le comparant à un référentiel de sources et aux travaux précédemment soumis au sein de l'institution. Cette fonctionnalité repose sur l'intégration d'algorithmes capables d'identifier non seulement les répétitions textuelles directes, mais aussi des formes de fraude plus complexes comme la reformulation. À l'issue de cette analyse, la plateforme doit produire un rapport de conformité mettant en évidence les segments nécessitant une attention particulière, permettant ainsi aux évaluateurs de statuer sur l'originalité du travail de manière objective.

Enfin, la gestion de la phase finale, incluant le dépôt définitif et la délibération, constitue un besoin fonctionnel essentiel. Le système doit permettre l'organisation numérique des étapes précédant la soutenance et la saisie sécurisée des évaluations par les membres du jury. Une fois le mémoire validé, Handal doit assurer son archivage automatique dans une bibliothèque numérique centralisée. L'ensemble de ces fonctionnalités concourt à créer un écosystème numérique fluide où la transparence et la rigueur sont maintenues depuis la définition du sujet jusqu'à l'archivage final du travail de recherche.

3. Spécification des besoins non-fonctionnels

Au-delà des fonctionnalités métiers, le système doit répondre à des exigences de qualité rigoureuses. La sécurité est la priorité absolue, impliquant des protocoles de communication chiffrés et une gestion des sessions robuste pour prévenir toute usurpation d'identité. La performance est également essentielle, notamment pour le moteur de détection qui doit traiter des volumes de données importants sans dégrader l'expérience utilisateur. Grâce à une architecture permettant le traitement asynchrone, les analyses textuelles s'exécutent en arrière-plan sans bloquer la navigation sur la plateforme.

L'évolutivité et la maintenabilité sont garanties par une architecture modulaire permettant d'ajouter de nouvelles fonctionnalités sans impacter l'existant. Le système doit être hautement disponible pour permettre un accès permanent aux dossiers, même lors des périodes de forte affluence. Le code est structuré de manière à faciliter les futures opérations de maintenance et l'intégration éventuelle avec d'autres outils institutionnels, assurant ainsi que la solution pourra s'adapter aux évolutions technologiques futures de l'établissement.

II. Modélisation UML (Approche 2TUP)

1. Capture des besoins fonctionnels (Cas d'utilisation)

La phase de conception adopte l'approche 2TUP, qui permet de séparer l'étude fonctionnelle des contraintes techniques. La capture des besoins fonctionnels s'attache à identifier les acteurs du système et les services qu'ils attendent de la plateforme. L'étudiant est l'acteur principal pour le dépôt et la consultation des rapports, tandis que l'enseignant intervient pour l'encadrement et la validation des étapes. L'administrateur, quant à lui, assure la gestion des comptes et la configuration des paramètres du moteur de détection. Cette modélisation permet de délimiter précisément le périmètre du projet.

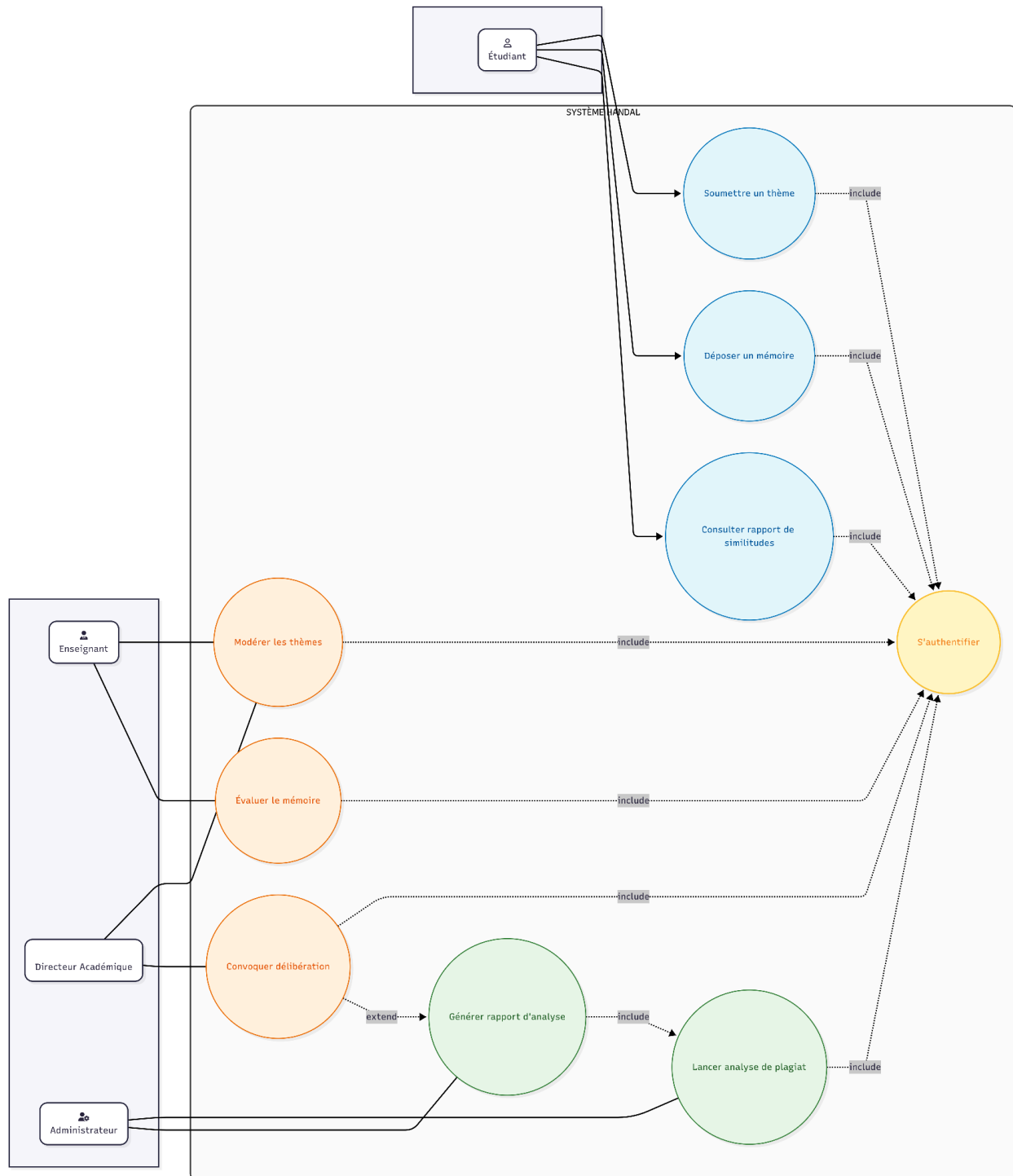


Figure 1: Diagramme de cas d'utilisation

2. Analyse dynamique (Diagrammes de séquence)

L'analyse dynamique détaille la chronologie des interactions entre les acteurs et le système pour les processus critiques. Le diagramme de séquence relatif à la soumission d'un mémoire illustre ainsi le cheminement du fichier depuis le navigateur de l'étudiant jusqu'au serveur de stockage, déclenchant automatiquement l'analyse de plagiat. Ce flux montre comment les composants logiciels communiquent entre eux pour valider le document, générer le rapport de similitudes et notifier l'encadreur en temps réel. Cette étape est cruciale pour valider la logique de communication et l'ordonnancement des tâches au sein de l'application.

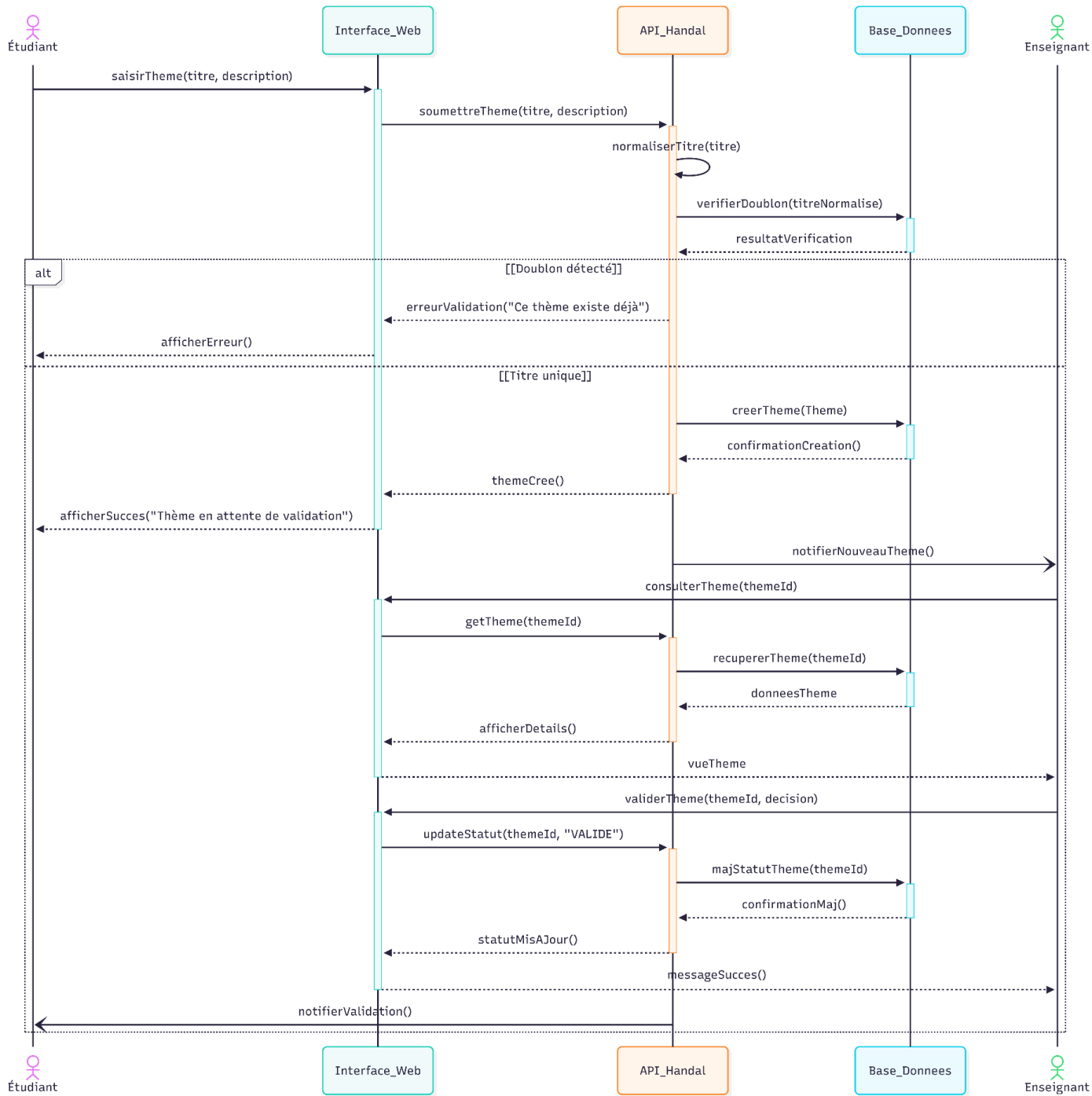


Figure 2: Diagramme de Séquence : Processus de soumission et validation de thème

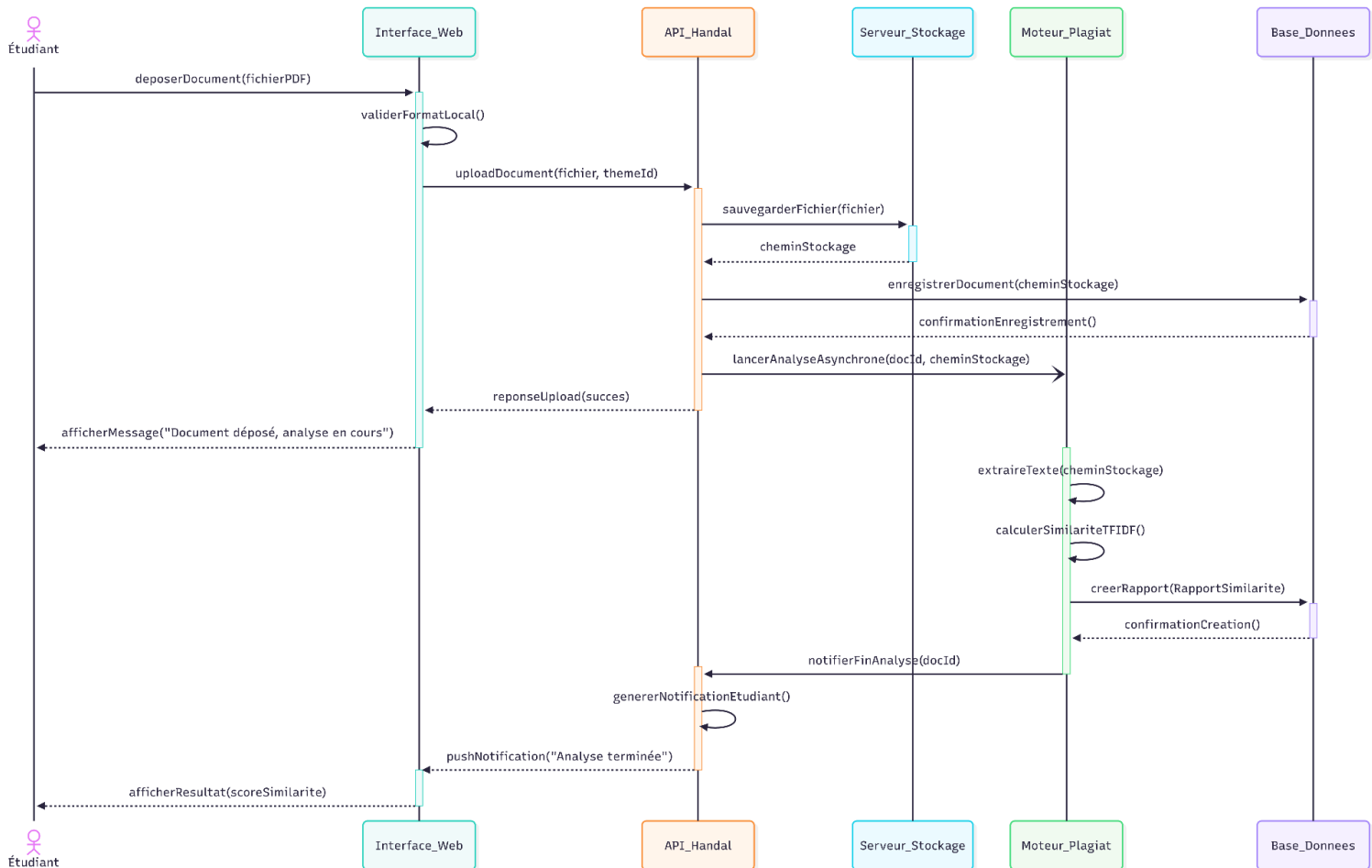


Figure 3: Diagramme de Séquence : Processus de dépôt et d'analyse automatique de mémoire

3. Analyse statique (Diagramme de classes)

Enfin, l'analyse statique définit la structure interne des données et les relations entre les différentes entités du système. Le diagramme de classes présente les objets fondamentaux tels que les utilisateurs, les mémoires, les rapports d'analyse et les sessions de soutenance. Il précise les attributs de chaque classe et les liens de dépendance ou d'association qui les unissent. Cette vue structurelle constitue le plan de construction de la base de données et des modèles logiciels, garantissant que

l'information est organisée de manière cohérente et optimisée pour les performances du moteur de recherche et de détection.

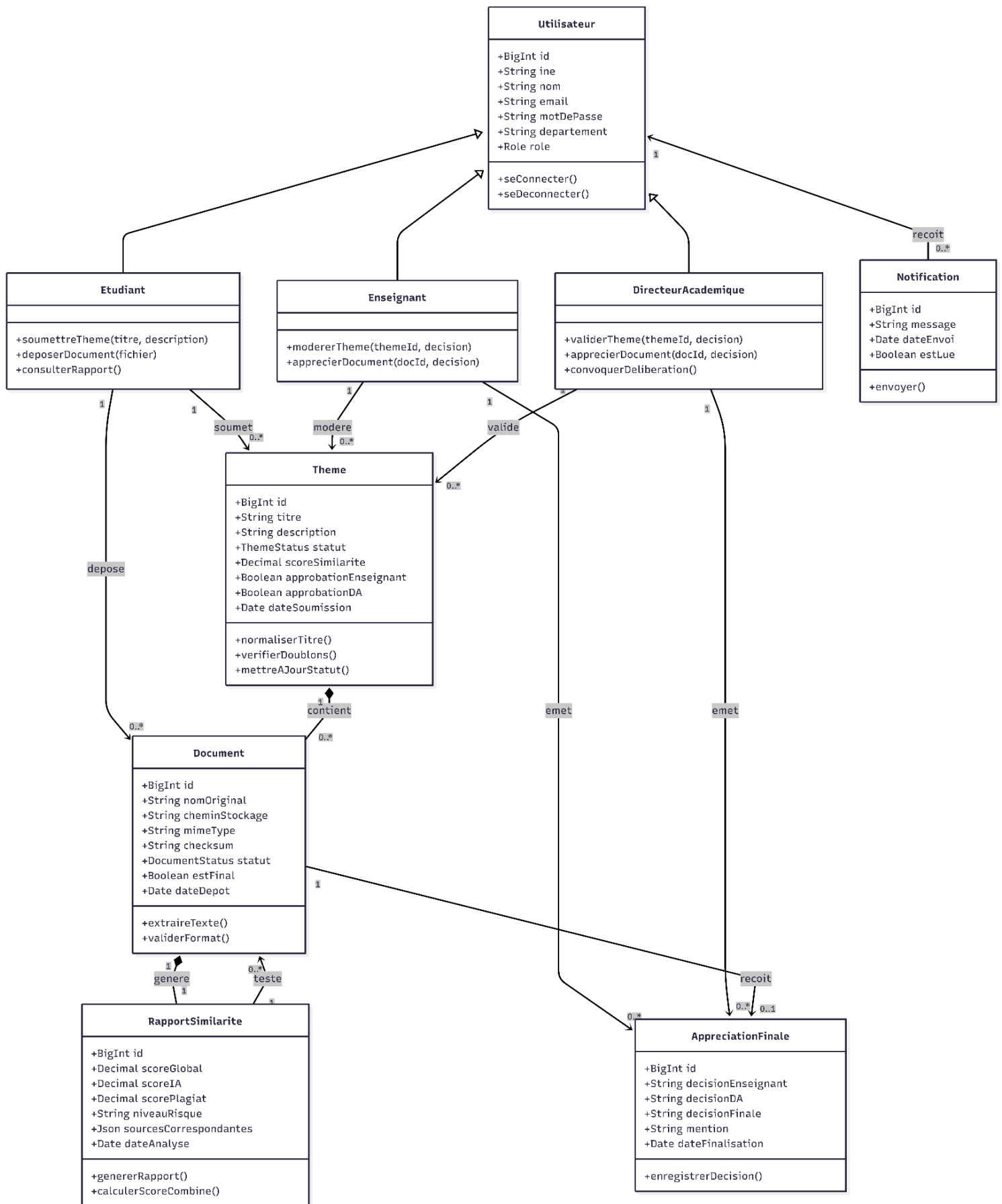


Figure 4: Diagramme de classe

CHAPITRE II : RÉALISATION ET MISE EN ŒUVRE

I. Environnement de développement

1. Choix technologiques

La réalisation de la plateforme Handal repose sur une pile technologique moderne et performante, sélectionnée pour répondre aux exigences d'interactivité et de traitement de données massives. Le choix s'est porté sur le framework Next.js, qui constitue la colonne vertébrale du projet. Cette solution offre une flexibilité totale grâce à son architecture full-stack, permettant de gérer à la fois le rendu côté serveur pour une meilleure optimisation et la création d'API robustes. En utilisant React pour la construction des interfaces, le système bénéficie d'une approche modulaire facilitant la maintenance et la réutilisation des composants, assurant ainsi une fluidité de navigation optimale pour les étudiants et le corps enseignant.

Pour l'aspect esthétique et l'ergonomie, Tailwind CSS a été privilégié afin de garantir un design moderne, épuré et entièrement réactif. Ce framework utilitaire permet une grande précision dans la mise en forme des interfaces tout en réduisant considérablement le poids des feuilles de style. La gestion de la persistance des données est confiée à Prisma ORM, qui assure une communication sécurisée et typée entre le serveur et la base de données MySQL. Ce choix technique garantit l'intégrité des données académiques et facilite les migrations structurelles au fil des évolutions du projet. Enfin, l'utilisation de TypeScript sur l'ensemble de la base de code sécurise le développement en limitant les erreurs d'exécution, renforçant ainsi la fiabilité globale du système de détection.

2. Environnement matériel et logiciel

Le développement et le déploiement de la solution s'appuient sur un écosystème matériel et logiciel performant, garantissant une productivité constante. Sur le plan logiciel, l'environnement de travail est centré sur l'éditeur Visual Studio Code, enrichi d'extensions spécialisées pour le débogage et le formatage du code. Le système d'exploitation Debian a été utilisé pour sa robustesse et sa gestion native des environnements de conteneurisation, facilitant ainsi la cohérence entre les phases de

développement local et la mise en production. La gestion de versions est assurée par Git, permettant une traçabilité complète des modifications et une collaboration efficace sur le code source.

Côté matériel, la conception du projet a été entièrement réalisée sur un ordinateur portable performant. Cette configuration mobile a permis de faire tourner simultanément les serveurs de développement, les bases de données locales et les modules de traitement du moteur d'analyse. Grâce à une gestion optimisée des ressources système et à l'utilisation d'un disque SSD à haute vitesse, les phases d'extraction de texte et de comparaison de fichiers ont pu être exécutées de manière fluide. Ce choix d'environnement démontre que l'architecture logicielle de la plateforme Handal est suffisamment optimisée pour être développée et testée sur une configuration standard, tout en restant capable de supporter des charges de travail intensives lors de son déploiement final.

II. Implémentation des fonctionnalités

1. Développement du moteur de détection (Plagiat & IA)

Le cœur technologique de la plateforme Handal réside dans son moteur d'analyse hybride, conçu pour identifier tant le plagiat classique que les contenus générés par intelligence artificielle. Pour l'extraction de données, le système intègre une bibliothèque de traitement de flux PDF permettant de convertir les documents déposés en texte brut normalisé. Une fois le texte extrait, le moteur procède à une analyse granulaire basée sur la méthode des n-grammes, découpant le contenu en séquences de mots pour les comparer à une base de données locale de référence. Cette comparaison permet de calculer un indice de similarité précis, identifiant les segments identiques ou fortement paraphrasés.

En complément, le module de détection d'IA s'appuie sur des algorithmes de traitement du langage naturel (NLP) qui analysent la structure syntaxique et la distribution statistique des termes. Contrairement à une simple recherche de mots-clés, le moteur évalue la perplexité et la régularité du texte, caractéristiques souvent révélatrices d'une production automatisée. Pour garantir l'intégrité du processus, chaque document se voit attribuer une empreinte numérique unique (Checksum SHA-256) dès son téléchargement. Cette mesure technique assure qu'aucun document identique ne puisse être soumis plusieurs fois et garantit que le fichier analysé est strictement celui déposé par l'étudiant.

2. Gestion du workflow de dépôt et validation

L'implémentation du workflow de Handal suit une logique rigoureuse de transition d'états, assurant une traçabilité complète du mémoire, du dépôt initial à la mention finale. Le processus débute par une interface de soumission intuitive où l'étudiant suit en temps réel l'évolution de son dossier grâce à une barre de progression dynamique. Dès que le fichier est téléversé, il est verrouillé en lecture seule, interdisant toute modification ultérieure pour préserver la validité de l'analyse. Le moteur de détection se déclenche alors automatiquement de manière asynchrone, permettant à l'utilisateur de poursuivre sa navigation pendant que les calculs intensifs s'exécutent en arrière-plan.

Une fois le rapport de similitudes généré, le workflow bascule vers la phase de validation académique. L'enseignant et le Directeur Académique accèdent à un tableau de bord dédié leur permettant de consulter les segments litigieux mis en évidence par le système. La validation est strictement conditionnée par la saisie d'une appréciation motivée : le système impose un commentaire justificatif pour chaque décision, qu'il s'agisse d'une validation ou d'un rejet pour réécriture. Ce processus collaboratif garantit que la décision finale ne repose pas uniquement sur un score automatisé, mais sur une expertise pédagogique humaine éclairée par les données techniques fournies par la plateforme.

III. Tests et Sécurité

1. Plan de tests (Unitaires et Intégration)

La fiabilité du système Handal repose sur une stratégie de tests rigoureuse, divisée en deux niveaux complémentaires. Les tests unitaires, réalisés principalement avec le framework Jest, ont permis de valider isolément chaque fonction critique du moteur de détection, notamment l'algorithme de comparaison par n-grammes et les modules de calcul de scores. Cette approche garantit que chaque brique logique produit un résultat mathématiquement exact avant son intégration. En parallèle, des scénarios de "tests aux limites" ont été conduits pour éprouver la robustesse du système face à des fichiers PDF corrompus ou des formats non conformes, assurant une gestion d'erreurs élégante sans interruption de service.

Les tests d'intégration ont quant à eux porté sur la communication entre les différentes couches de l'application. Une attention particulière a été accordée à la liaison entre les routes d'API Next.js et la couche d'accès aux données gérée par Prisma. Ces tests ont permis de vérifier la cohérence des transactions en base de données, s'assurant par exemple qu'un rapport de similitude est systématiquement lié au bon document et au bon étudiant. Cette phase de validation de bout en bout garantit que le workflow, de la soumission à la délibération, fonctionne de manière fluide et sans perte d'intégrité de l'information.

2. Mesures de sécurité et protection des données

La sécurité de la plateforme Handal est articulée autour de la protection de l'identité et de la confidentialité des travaux académiques. L'authentification des utilisateurs est sécurisée par le hachage systématique des mots de passe via l'algorithme bcrypt, rendant les données illisibles même en cas d'accès non autorisé à la base de données. L'accès aux ressources est strictement contrôlé par des sessions sécurisées, garantissant qu'un étudiant ne peut accéder qu'à ses propres rapports, tandis que les enseignants sont limités aux documents de leur département respectif.

Concernant la protection des documents, plusieurs barrières techniques ont été instaurées. Les fichiers stockés sur le serveur sont renommés à l'aide de leur empreinte numérique (SHA-256) pour éviter toute fuite d'information par devinette d'URL et pour garantir l'unicité des dépôts. De plus, toutes les configurations sensibles (identifiants de base de données, clés secrètes) sont isolées dans des variables d'environnement non exposées au code client. Enfin, pour respecter la confidentialité des recherches, le système prévoit une gestion du cycle de vie des données permettant la suppression définitive des fichiers et des extractions de texte une fois les processus de délibération officiellement clos.

CONCLUSION

Le présent projet de fin de cycle a consisté en la conception et la réalisation de Handal, une plateforme web innovante de gestion des mémoires et de détection automatique de plagiat et de contenus générés par IA. À travers une démarche structurée, nous avons d'abord analysé les besoins spécifiques des étudiants et du corps enseignant pour aboutir à une architecture logicielle moderne basée sur l'écosystème Next.js. Le cœur de notre travail a porté sur l'implémentation d'un moteur d'analyse capable d'extraire et de comparer des données textuelles de manière asynchrone, tout en assurant une expérience utilisateur fluide et sécurisée. Ce projet a permis de transformer un besoin académique en une solution technique fonctionnelle, intégrant des problématiques de développement full-stack, de traitement du langage naturel et de gestion rigoureuse de workflows.

Au terme de ce développement, les principaux objectifs fixés ont été atteints : la plateforme permet désormais une soumission simplifiée des travaux, une détection précise des similitudes via une base de données locale et un processus de validation transparent pour les autorités académiques. L'utilisation de technologies comme Prisma et Tailwind CSS a permis d'obtenir un outil robuste, sécurisé et visuellement ergonomique.

Toutefois, le projet a rencontré certaines limites inhérentes à un développement en cycle court. Le moteur de détection, bien qu'efficace sur les textes bruts, reste limité face à des documents contenant des images ou des schémas complexes non textuels. De plus, la détection d'IA, bien que fonctionnelle, demeure un défi technique constant en raison de l'évolution rapide des modèles de génération de langage, ce qui nécessite une mise à jour régulière des algorithmes de détection. Enfin, l'absence actuelle d'interconnexion avec des bases de données de travaux universitaires internationaux limite le champ de comparaison aux documents déposés localement.

Le projet Handal est conçu pour être évolutif. À court terme, l'intégration d'un module d'OCR (Reconnaissance Optique de Caractères) permettrait d'analyser les mémoires numérisés ou contenant des captures de texte. Une autre perspective majeure réside dans le déploiement d'une architecture distribuée via des conteneurs Docker, afin de supporter une augmentation massive du nombre d'utilisateurs simultanés.

À plus longue échelle, nous envisageons d'enrichir le moteur de détection par l'intégration d'API de recherche globales pour comparer les mémoires aux milliards de pages disponibles sur le web, augmentant ainsi considérablement la pertinence du score de similitude. L'objectif ultime est de faire de Handal un standard académique au sein de l'institut, en y ajoutant des fonctionnalités de gestion de jurys et d'archivage numérique pérenne pour les promotions futures.